PaXini

# API 参考 Python



PaXini Tech

# GMH15



# CONTENTS

PaXini


1. 控制类 DexH15Control — 01

2. 相机控制类 DexH15Camera — 34

3. 运动学解析类 DexH15Kinematic — 37

4. 关节顺序定义 — 40

5. 数据结构 — 41

6. 枚举类型 — 44

# 1. 控制类 DexH15Control

负责灵巧手的串口通信、电机控制、传感器读取、力控、动作管理、故障诊断与固件升级等核心操作。
构造函数: ctrl = DexH15Control(), 无参数。

## 1.1 connectModbusAuto

- **功能概述**

  自动连接当前串口下的第一个灵巧手设备。适用于单设备场景, 系统将自动完成设备发现与连接流程。

- **函数原型**

```
def connectModbusAuto(self, hand_port_num: str) -> tuple[bool, int]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| hand_port_num | str | 输入 | 灵巧手连接的串口编号 (如: "/dev/ttyUSB0") |

- **返回值**

  成功时返回一个 tuple[bool, int], 包含两个元素:
  - result[0] (bool): 是否连接成功
  - result[1] (int): 成功连接后返回的第一个设备地址

- **特别说明**

  程序执行过程中发生错误, 则会抛出异常。

## 1.2 openModbusDevice

- **功能概述**

  打开指定串口设备, 建立 Modbus 通信连接。

- **函数原型**

```
def openModbusDevice(self, hand_port_num: str, baud_rate: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| hand_port_num | str | 输入 | 灵巧手连接的串口编号 |
| baud_rate | int | 输入 | 串口设备通讯的波特率: 4000000, 921600, 460800, 230400, 115200, 57600 |

- **返回值**

  - True: 打开成功
  - False: 打开失败



## 1.3 openUsbDevice

- **功能概述**

  打开 USB 串口设备并建立连接, 自动获取从站地址。

- **函数原型**

```
def openUsbDevice(self, hand_port_num: str) -> tuple[bool, int]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| hand_port_num | str | 输入 | USB 设备编号 |

- **返回值**

  返回 result: tuple[bool, int]:
  - result[0] (bool): 是否打开成功
  - result[1] (int): 成功连接后返回的从站地址

## 1.4 scanModbusDevices

- **功能概述**

  扫描已打开串口下所有可通讯的设备地址。

- **函数原型**

```
def scanModbusDevices(self, hand_port_num: str) -> list[int]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| hand_port_num | str | 输入 | 灵巧手连接的串口编号 |

- **返回值**

  返回已打开串口下所有可通讯设备地址的集合 (list[int])。

## 1.5 initModbusDevice

- **功能概述**

初始化灵巧手 Modbus 设备配置。

- **函数原型**

```
def initModbusDevice(self, slave_address: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

- 1: 初始化成功
- 0: 初始化失败

## 1.6 disconnectModbus

- **功能概述**

关闭当前 Modbus 串口连接, 释放串口资源。

- **函数原型**

```
def disconnectModbus(self) -> bool
```

- **参数说明**

无

- **返回值**

- True: 断开成功
- False: 断开失败



## 1.7 disconnectUsb

- **功能概述**

关闭 USB 设备连接, 释放相关资源。

- **函数原型**

```
def disconnectUsb(self) -> bool
```

- **参数说明**

无

- **返回值**

- True: 操作执行完成

## 1.8 isModbusDeviceConnected

- **功能概述**

检查指定从站地址的灵巧手 Modbus 连接状态。

- **函数原型**

```
def isModbusDeviceConnected(self, slave_address: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

- True: 设备已连接
- False: 设备未连接

## 1.9 setModbusBaudrate

- **功能概述**

修改 Modbus 从站的设备地址。

- **函数原型**

```
def setModbusBaudrate(self, slave_address: int, baud_rate: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| baud_rate | int | 输入 | 目标波特率: 4000000, 921600, 460800, 230400, 115200, 57600 |

- **返回值**

- True: 设置成功
- False: 设置失败

- **特别说明**

波特率的设置必须在使用串口前完成，设置完成后需重启设备方可生效。同一串口上挂载的所有设备波特率必须保持一致。

## 1.10 getModbusBaudrate

- **功能概述**

获取当前 Modbus 从站的通信波特率。

- **函数原型**

```
def getModbusBaudrate(self, slave_address: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回从站设备当前的波特率值（int）。



## 1.11 setModbusDeviceAddress

- **功能概述**

修改 Modbus 从站的设备地址。

- **函数原型**

```
def setModbusDeviceAddress(self, slave_address: int, new_address: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 当前从站地址（0x01 ~ 0xF7） |
| new_address | int | 输入 | 新的从站地址（0x01 ~ 0xF7） |

- **返回值**

- True: 设置成功
- False: 设置失败

- **特别说明**

从站地址的设置必须在使用前完成，设置完成后需重启设备方可生效。

## 1.12 getModbusConfig

- **功能概述**

获取当前从站设备的 Modbus 通信配置信息。

- **函数原型**

```
def getModbusConfig(self, slave_address: int) -> ModbusConfig
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 ModbusConfig 结构体，包含 baud_rate（波特率）、slave_id（从机 ID）字段。

## 1.13 enableMotor

- **功能概述**

使能（上电激活）指定从站的所有电机。

- **函数原型**

```
1  def enableMotor(self, slave_address: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

- True: 使能成功
- False: 使能失败

## 1.14 disableMotor

- **功能概述**

下使能（断电）指定从站的所有电机。

- **函数原型**

```
1  def disableMotor(self, slave_address: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

- True: 下使能成功
- False: 下使能失败



## 1.15 isMotorEnabled

- **功能概述**

查询指定从站电机的当前使能状态。

- **函数原型**

```
1  def isMotorEnabled(self, slave_address: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

- True: 电机已使能
- False: 电机未使能

## 1.16 initMotorPosition

- **功能概述**

初始化所有电机位置，使灵巧手回到初始零位状态。

- **函数原型**

```
1  def initMotorPosition(self, slave_address: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

- 1: 初始化成功
- 0: 初始化失败

## 1.17 getMotorMaxVelocityConfig

- **功能概述**

设置电机的控制模式。默认模式为位置模式，可根据需要切换为电流模式或速度模式。

- **函数原型**

```
1  def setMotorControlMode(self, slave_address: int, mode: ControlMode) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| mode | ControlMode | 输入 | 电机控制模式枚举值 |

- **ControlMode 枚举值说明**

| 枚举值 | 说明 |
| --- | --- |
| NONE | 无效值 |
| CURRENT_CONTROL_MODE | 电流环控制模式 |
| VELOCITY_CONTROL_MODE | 速度环控制模式 |
| POSITION_CONTROL_MODE | 位置环控制模式（默认） |

- **返回值**

- True: 设置成功
- False: 设置失败

## 1.18 getMotorControlMode

- **功能概述**

获取当前电机的控制模式。

- **函数原型**

```
def getMotorControlMode(self, slave_address: int) -> ControlMode
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 ControlMode 枚举值:

| 枚举值 | 说明 |
| --- | --- |
| NONE | 无效值（仅在获取失败时出现） |
| CURRENT_CONTROL_MODE | 电流环控制模式 |
| VELOCITY_CONTROL_MODE | 速度环控制模式 |
| POSITION_CONTROL_MODE | 位置环控制模式 |

## 1.19 setJointPositionsAngle（全关节）

- **功能概述**

控制全部 11 个手指关节的目标角度（归一化角度），关节顺序见「4. 关节顺序定义」。

- **函数原型**

```
def setJointPositionsAngle(self, slave_address: int, joint_angles: list[float]) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int16_t | 输入 | 从站地址 |
| joint_angles | list[float] | 输入 | 11 个可调关节的归一化角度值集合 |

- **返回值**

- 1: 设置成功
- 0: 设置失败



## 1.20 setJointPositionsAngle（指定手指）

- **功能概述**

设置指定单根手指的关节归一化角度。

- **函数原型**

```
def setJointPositionsAngle(self, slave_address: int, finger_id: FingerType, joint_angles: list[float]) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int16_t | 输入 | 从站地址 |
| finger_id | FingerType | 输入 | 目标手指的枚举值 |
| joint_angles | list[float] | 输入 | 指定手指的归一化角度值集合 |

- **FingerType 枚举值定义**

| 枚举值 | 数值 | 说明 |
| --- | --- | --- |
| INVALID_FINGER | 0 | 无效值 |
| INDEX_FINGER | 1 | 食指 |
| MIDDLE_FINGER | 2 | 中指 |
| RING_FINGER | 3 | 无名指 |
| PINKY_FINGER | 4 | 小拇指 |
| THUMB_FINGER | 5 | 大拇指 |

- **返回值**

- 1: 设置成功
- 0: 设置失败

## 1.21 getJointPositionsAngle（全关节）

- **功能概述**

获取全部手指关节的当前归一化角度。当 is_all_joint 为 false 时获取 11 个可调关节角度; 为 true 时额外返回 4 个末端指骨（DIP）关节, 共获取 15 个关节角度。

- **函数原型**

```
def getJointPositionsAngle(self, slave_address: int, is_all_joint: bool = False) -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| is_all_joint | bool | 输入 | 是否获取全部关节角度, 默认 False（获取 11 个可调关节） |

- **返回值**

返回 tuple[int, list[float]]:
- result[0] int: 1 成功 / 0 失败
- result[1] list[float]: is_all_joint=False 时长度为 11, True 时长度为 15（关节顺序同 C++ 版 1.32 节）

## 1.22 getJointPositionsAngle（指定手指）

- **功能概述**

获取指定单根手指的当前关节角度。当 is_all_joint 为 false 时该手指返回 2 个关节（大拇指为 3 个）；为 true 时该手指额外返回 1 个末端指骨（DIP）关节。

- **函数原型**

```
def getJointPositionsAngle(self, slave_address: int, finger_id: FingerType,
                           is_all_joint: bool = False) -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
|---|---|---|---|
| slave_address | int | 输入 | 从站地址 |
| finger_id | FingerType | 输入 | 目标手指的枚举值 |
| is_all_joint | bool | 输入 | 是否获取该手指全部关节角度，默认 False |

- **返回值**

返回 tuple[int, list[float]]:

- result[0] int: 1 成功 / 0 失败
- result[1] list[float]: 该手指的关节角度值

## 1.23 setMotorTargetPosition（全部电机）

- **功能概述**

通过电机位置参数结构体，统一设置所有电机的目标位置。

- **函数原型**

```
def setMotorTargetPosition(self, slave_address: int, target_position: Dex15MotorPosition) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
|---|---|---|---|
| slave_address | int16_t | 输入 | 从站地址 |
| target_position | Dex15MotorPosition | 输入 | 7 路电机的目标位置值 |

- **返回值**

- 1: 设置成功
- 0: 设置失败



## 1.24 setMotorTargetPosition（指定电机）

- **功能概述**

设置指定单个电机的目标位置。

- **函数原型**

```
def setMotorTargetPosition(self, slave_address: int, motor_id: FingerMotor, target_position: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
|---|---|---|---|
| slave_address | int | 输入 | 从站地址 |
| motor_id | FingerMotor | 输入 | 目标电机的枚举索引值（见 6.4 FingerMotor） |
| target_position | int | 输入 | 目标电机位置值 |

- **返回值**

- 1: 设置成功
- 0: 设置失败

## 1.25 getMotorPosition

- **功能概述**

获取当前所有电机的位置信息。

- **函数原型**

```
def getMotorPosition(self, slave_address: int) -> Dex15MotorPosition
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
|---|---|---|---|
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 Dex15MotorPosition 结构体，包含所有电机的当前位置信息（motor1_pos ~ motor7_pos）。

## 1.26 getJointMagneticEncoder

- **功能概述**

获取当前从站设备所有关节的磁编码器原始数据。

- **函数原型**

```
1  def getJointMagneticEncoder(self, slave_address: int) -> list[int]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回包含所有关节磁编码器数值的列表（list[int]）。

## 1.27 setMotorTargetVelocity（全部电机）

- **功能概述**

在速度模式下，统一设置所有电机的目标速度，用于控制手的整体运动速度。

- **函数原型**

```
1  def setMotorTargetVelocity(self, slave_address: int, target_speed: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| target_speed | int | 输入 | 全部电机的统一目标速度，取值范围 [-1000, 1000] |

- **返回值**

- 1：设置成功
- 0：设置失败



## 1.28 setMotorTargetVelocity（全部电机）

- **功能概述**

在速度控制模式下，设置指定单个电机的目标速度。

- **函数原型**

```
1  def setMotorTargetVelocity(self, slave_address: int, motor_id: FingerMotor, target_speed: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| motor_id | FingerMotor | 输入 | 目标电机的枚举索引值（见 6.4 FingerMotor） |
| target_speed | int | 输入 | 目标速度值，取值范围 [-1000, 1000] |

- **返回值**

1：设置成功
0：设置失败

## 1.29 getMotorActualVelocity

- **功能概述**

获取当前所有电机的实际运行速度。

- **函数原型**

```
1  def getMotorActualVelocity(self, slave_address: int) -> tuple[int, Dex15MotorSpeed]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 result: tuple[int, Dex15MotorSpeed]:

- result[0]（int）：状态码，1 获取成功 / 0 获取失败
- result[1]（Dex15MotorSpeed）：7 路电机实际速度，范围 [-1000, 1000]

## 1.30 setMotorMaxVelocity

- **功能概述**

设置所有电机的最大速度限制。

- **函数原型**

```
1  def setMotorMaxVelocity(self, slave_address: int, max_speed: Dex15MotorSpeed) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| max_speed | Dex15MotorSpeed | 输入 | 各电机最大速度配置，取值范围 [0, 1000] |

- **返回值**

- True: 设置成功
- False: 设置失败

## 1.31 getMotorMaxVelocityConfig

- **功能概述**

获取当前从站设备所有电机的最大速度配置。

- **函数原型**

```
1  def getMotorMaxVelocityConfig(self, slave_address: int) -> Dex15MotorSpeed
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 Dex15MotorSpeed 结构体，包含所有电机的最大速度配置。



## 1.32 setMotorTargetCurrent（全部电机）

- **功能概述**

在电流模式下，设置所有电机的目标电流。目标电流值越大，手的握力越强。

- **函数原型**

```
1  def setMotorTargetCurrent(self, slave_address: int, target_current: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| target_current | int | 输入 | 全部电机的统一目标电流，取值范围 [-1000, 1000] |

- **返回值**

- 1: 设置成功
- 0: 设置失败

## 1.33 setMotorTargetCurrent（指定电机）

- **功能概述**

在电流模式下，设置指定单个电机的目标电流。

- **函数原型**

```
1  def setMotorTargetCurrent(self, slave_address: int, motor_id: FingerMotor, target_current: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| motor_id | FingerMotor | 输入 | 目标电机的枚举索引值（见 6.4 FingerMotor） |
| target_current | int | 输入 | 目标电流值，取值范围 [-1000, 1000] |

- **返回值**

- 1: 设置成功
- 0: 设置失败

## 1.34 getMotorActualCurrent

- **功能概述**

获取当前电机的控制模式。

- **函数原型**

```
1  def getMotorActualCurrent(self, slave_address: int) -> tuple[int, Dex15MotorCurrent]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 tuple[int, Dex15MotorCurrent]:

- result[0] (int): 状态码, 1 获取成功 / 0 获取失败
- result[1] (Dex15MotorCurrent): 7 路电机实际电流, 范围 [-1000, 1000]

## 1.35 setMotorMaxCurrent

- **功能概述**

设置所有电机的最大电流, 用于限制握力峰值。

- **函数原型**

```
1  def setMotorMaxCurrent(self, slave_address: int, max_current: Dex15MotorCurrent) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| max_current | Dex15MotorCurrent | 输入 | 各电机最大电流配置, 取值范围 [0, 1000] |

- **返回值**

- True: 设置成功
- False: 设置失败



## 1.36 getMotorMaxCurrentConfig

- **功能概述**

获取当前从站设备所有电机的最大电流配置。

- **函数原型**

```
1  def getMotorMaxCurrentConfig(self, slave_address: int) -> Dex15MotorCurrent
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 Dex15MotorCurrent 结构体, 包含所有电机的最大电流配置。

## 1.37 getFingerTactile [deprecated]

- **功能概述**

【已弃用】此函数已不再维护, 请改用 getFingerResultantForce。获取全部手指触觉传感器的合力信息。手共有 5 个传感器, 每个传感器返回一个三轴合力。

- **函数原型**

```
1  def getFingerTactile(self, slave_address: int) -> list[ForcePoint]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回包含 5 个 ForcePoint 的列表, 每个 ForcePoint 包含:

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| x | int | 传感器 X 轴力分量, 单位 0.1N |
| y | int | 传感器 Y 轴力分量, 单位 0.1N |
| z | int | 传感器 Z 轴力分量, 单位 0.1N |

## 1.38 getFingerResultantForce

- **功能概述**

获取全部手指触觉传感器的合力信息。手共有 5 个传感器，每个传感器返回一个三轴合力。

- **函数原型**

```
1  def getFingerResultantForce(self, slave_address: int) -> list[ForcePoint]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回包含 5 个 ForcePoint 的列表，每个含 x、y、z 三轴分量，单位 0.1N。

## 1.39 getFingerTactileDetail [deprecated]

- **功能概述**

【已弃用】此函数已不再维护，请改用 getFingerDistributedForce。获取全部手指触觉传感器的分布力详细信息。

- **函数原型**

```
1  def getFingerTactileDetail(self, slave_address: int) -> list[ForcePoint]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回包含 327 个 ForcePoint 的列表。分布规则如下：

| 手指 | 分布力点数 |
| --- | --- |
| 食指 | 50 |
| 中指 | 50 |
| 无名指 | 50 |
| 小拇指 | 50 |
| 大拇指 | 127 |

每个 ForcePoint 包含 x、y、z 三轴分量，单位 0.1N。



## 1.40 getFingerDistributedForce

- **功能概述**

获取全部手指的分布式触觉力语义与 getFingerTactileDetail 一致获取全部手指触觉传感器的分布力详细信息。

- **函数原型**

```
1  def getFingerDistributedForce(self, slave_address: int) -> list[ForcePoint]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回分布式力点列表（共 327 个 ForcePoint），每个含 x、y、z 三轴分量，单位 0.1N。

## 1.41 getFingerTactileData

- **功能概述**

一次性获取指定单根手指的合力、分布力以及温度数据。

- **函数原型**

```
1  def getFingerTactileData(self, slave_address: int, finger_id: FingerType) -> tuple[ForcePoint, list[ForcePoint], FingerTemp]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| finger_id | FingerType | 输入 | 目标手指的枚举类型值（见 1.20） |

- **返回值**

返回 tuple[ForcePoint, list[ForcePoint], FingerTemp]:
- result[0]（ForcePoint）：该手指的触觉传感器合力信息
- result[1]（list[ForcePoint]）：该手指的触觉传感器分布力信息
- result[2]（FingerTemp）：该手指的传感器温度数据

## 1.42 getAllTactileData

- **功能概述**

一次性获取所有手指的合力、分布力以及温度数据。

- **函数原型**

```
1  def getAllTactileData(self, slave_address: int) -> tuple[list[ForcePoint], list[ForcePoint], list[FingerTemp]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 result: tuple[list[ForcePoint], list[ForcePoint], list[FingerTemp]]:
- result[0] (list[ForcePoint]) : 全部手指的合力数据 (5 个)
- result[1] (list[ForcePoint]) : 全部手指的分布力数据 (327 个)
- result[2] (list[FingerTemp]) : 全部手指的温度数据 (5 个)

## 1.43 getPressPositionForce

- **功能概述**

获取当前所有手指按压力的位置力点集合, 单位为 0.1N。

- **函数原型**

```
1  def getPressPositionForce(self, slave_address: int) -> list[PressForcePoint]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 list[PressForcePoint], 每项字段:

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| x / y / z | int | 坐标位置 |
| force_value | int | 力值大小, 单位 0.1N |
| model_name | str | 模型名称 |



## 1.44 getSDKVersion

- **功能概述**

获取全部手指传感器的温度数据。手共有 5 个传感器, 返回 5 个温度读数。

- **函数原型**

```
1  def getTemperature(self, slave_address: int) -> list[FingerTemp]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回包含 5 个 FingerTemp 的列表, 每个 FingerTemp 包含:

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| finger_name | str | 手指名称 |
| sensor_ID | int | 传感器 ID |
| temperature | float | 传感器温度值 ( 单位 ℃) |

## 1.45 calibrateSensor

- **功能概述**

触觉传感器标定。

- **函数原型**

```
1  def calibrateSensor(self, slave_address: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int16_t | 输入 | 从站地址 |

- **返回值**

- True: 标定成功
- False: 标定失败

## 1.46 setCommonForceCtlParam

- **功能概述**

配置普通力控的初始位置和控制参数。配置完成后，通过 computeCommonForceTargetCommand 计算每次运动的目标指令。

- **函数原型**

```
1  def setCommonForceCtlParam(self, slave_address: int, joint_angles: list[float], param: ForceCtlParam) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| joint_angles | list[float] | 输入 | 普通力控的初始灵巧手位置，需传入 11 个可调关节的归一化角度值 |
| param | ForceCtlParam | 输入 | 力控参数配置（使能标志 + 阻抗系数 + 参考力系数），不设置时使用默认参数 |

- **返回值**

- 1: 配置成功
- 0: 配置失败

- **特别说明**

- 普通力控与切向力力控的参数配置会相互覆盖，两者不可同时设置使用。被覆盖后需重新设置参数，才能进行目标位置的计算。
- 灵巧手的当前位置与需要进行力控的初始位置 joint_angles 不同时,需要视情况休眠等待灵巧手到达初始位置。

## 1.47 setMotorTargetPosition（指定电机）

- **功能概述**

在普通力控模式下，根据当前力传感器数据计算下一次运动的目标关节角度指令。

- **函数原型**

```
1  def computeCommonForceTargetCommand(self, slave_address: int) -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 result: tuple[int, list[float]]:
- result[0]（int）: 状态码，1 计算成功 / 0 计算失败
- result[1]（list[float]）: 下次运动的归一化角度指令，包含 11 个可调关节数据



## 1.48 setTangentialForceCtlParam

- **功能概述**

配置切向力力控的初始位置和控制参数。配置完成后，通过 computeTangentialForceTargetCommand 计算每次运动的目标指令。

- **函数原型**

```
1  def setTangentialForceCtlParam(self, slave_address: int, joint_angles: list[float], param: TanForceCtlParam) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| joint_angles | list[float] | 输入 | 切向力力控的初始灵巧手位置，需传入 11 个可调关节的归一化角度值 |
| param | TanForceCtlParam | 输入 | 切向力力控参数配置（含切向力增益），不设置时使用默认参数 |

- **返回值**

- 1: 配置成功
- 0: 配置失败

- **特别说明**

- 普通力控与切向力力控的参数配置会相互覆盖，两者不可同时设置使用。被覆盖后需重新设置参数，才能进行目标位置的计算。
- 灵巧手的当前位置与需要进行力控的初始位置 joint_angles 不同时,需要视情况休眠等待灵巧手到达初始位置。

## 1.49 computeTangentialForceTargetCommand

- **功能概述**

在切向力力控模式下，根据当前力传感器数据计算下一次运动的目标关节角度指令。

- **函数原型**

```
1  def computeTangentialForceTargetCommand(self, slave_address: int) -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int16_t | 输入 | 从站地址 |

- **返回值**

返回 result: tuple[int, list[float]]:
- result[0]（int）: 状态码，1 计算成功 / 0 计算失败
- result[1]（list[float]）: 下次运动的归一化角度指令，包含 11 个可调关节数据

## 1.50 calculateRealAngle

- **功能概述**

根据归一化角度值, 计算对应的真实物理角度 ( 单位: 度 )。

- **函数原型**

```
1  def calculateRealAngle(self, slave_address: int, normalize_angle: list[float]) -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| normalize_angle | list[float] | 输入 | 11 个可调关节的归一化角度值集合( 范围见 4. 关节顺序定义 - 归一 化范围列 ) |

- **返回值**

返回 result: tuple[int, list[float]]:
- result[0] ( int ) : 状态码, 1 计算成功 / 0 计算失败
- result[1] ( list[float] ) : 计算得到的 11 个关节真实角度值集合 ( 单位: °)

## 1.51 calculateNormalizeAngle

- **功能概述**

根据真实物理角度值, 计算对应的归一化角度。

- **函数原型**

```
1  def calculateNormalizeAngle(self, slave_address: int, real_angle: list[float]) -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| real_angle | list[float] | 输入 | 11 个可调关节的真实角度值集合 ( 单位: °) |

- **返回值**

返回 result: tuple[int, list[float]]:
- result[0] ( int ) : 状态码, 1 计算成功 / 0 计算失败
- result[1] ( list[float] ) : 计算得到的 11 个可调关节归一化角度值集合



## 1.52 saveCustomizedAction ( 指定动作帧 )

- **功能概述**

将指定的电机位置数据保存为用户自定义动作。

- **函数原型**

```
1  def saveCustomizedAction(self, slave_address: int, index: int, action: Dex15MotorPosition) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| index | int | 输入 | 自定义动作序列编号, 范围 [1, 20] |
| action | Dex15MotorPosition | 输入 | 要保存的 7 路电机位置数据 |

- **返回值**

- 1: 保存成功
- 0: 保存失败

## 1.53 saveCustomizedAction ( 当前位置 )

- **功能概述**

将灵巧手当前实时位置直接保存为指定编号的自定义动作。

- **函数原型**

```
1  def saveCustomizedAction(self, slave_address: int, index: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| index | int | 输入 | 自定义动作序列编号, 范围 [1, 20] |

- **返回值**

- 1: 保存成功
- 0: 保存失败

## 1.54 saveCustomizedAction（关节角度）

- **功能概述**

根据指定的关节归一化角度集合，计算电机位置后保存为自定义动作。

- **函数原型**

```
def saveCustomizedAction(self, slave_address: int, index: int, joint_angle_list: list[float]) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| index | int | 输入 | 自定义动作序列编号，范围 [1, 20] |
| joint_angle_list | list[float] | 输入 | 要保存的关节真实角度集合（11 个可调关节） |

- **返回值**

- 1：保存成功
- 0：保存失败

## 1.55 getActionPosition

- **功能概述**

加载并读取内置或自定义动作序列对应的电机位置信息。

- **函数原型**

```
def getActionPosition(self, slave_address: int, index: int, is_customized: bool = True) -> tuple[int, Dex15MotorPosition]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| index | int | 输入 | 动作序列编号（is_customized=True 时范围 [1, 20]; is_customized=False 时范围 [1, 10]） |
| is_customized | bool | 输入 | True: 自定义动作; False: 官方内置动作（默认 True） |

- **返回值**

返回 tuple[int, Dex15MotorPosition]:
- result[0]（int）：状态码，1 获取成功 / 0 获取失败
- result[1]（Dex15MotorPosition）：获取到的电机位置数据



## 1.56 runAction

- **功能概述**

执行指定编号的内置或自定义动作。

- **函数原型**

```
def runAction(self, slave_address: int, index: int, is_customized: bool = True) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| index | int | 输入 | 动作序列编号（is_Customized=True 时范围 [1, 20]; is_Customized=False 时范围 [1, 10]） |
| is_customized | bool | 输入 | True: 自定义动作; False: 官方内置动作（默认 True） |

- **返回值**

1：执行成功
0：执行失败

## 1.57 checkHandStatus

- **功能概述**

检查灵巧手的整体运行状态。

- **函数原型**

```
def checkHandStatus(self, slave_address: int) -> int
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

| 返回值 | 说明 |
| --- | --- |
| 1 | 正常运行 |
| 2 | 容错运行状态（出现了错误，但不影响核心接口使用） |
| 3 | 不可运行状态 |

## 1.58 getFaultCode

- **功能概述**

获取当前电机的故障码，用于诊断电机故障原因。

- **函数原型**

```
1  def getFaultCode(self, slave_address: int) -> list[FingerMotorsError]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 list[FingerMotorsError] 故障码列表，每项字段：

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| error_code | ErrorCode | 错误码枚举 |
| finger_name | str | 手指名称 |
| motor_ID | int | 电机 ID |

- **ErrorCode 故障码对照表**

| 错误名 | 错误码 | 错误描述 | 解释说明 | 解决办法 |
| --- | --- | --- | --- | --- |
| OVER_VOLT | 0 | 手指电机过压 | 驱动供电电压高于 15V 持续 2 秒 | 电压恢复后执行清除错误指令即可 |
| UNDER_VOLT | 1 | 手指电机欠压 | 驱动供电电压低于 10V 持续 2 秒 | 电压恢复后执行清除错误指令即可 |
| OVER_CURRENTU | 2 | U 相过流 | U 相电流过大 | 电流恢复正常后执行清除错误指令即可 |
| OVER_CURRENTV | 3 | V 相过流 | V 相电流过大 | 电流恢复正常后执行清除错误指令即可 |
| OVER_CURRENTW | 4 | W 相过流 | W 相电流过大 | 电流恢复正常后执行清除错误指令即可 |
| OVER_TEMPERATURE | 5 | 手指电机过温 | 电机温度过高 | 待温度恢复正常范围后执行清除错误指令即可 |
| ADC_CALIB | 6 | 硬件自检错误 | AD 静态偏差 +10% | 检查驱动磁场环境排除干扰后断电重启；无效则更换驱动板 |
| FOC_ERR | 7 | FOC 计算错误 | 无 | 无 |
| OVER_IQ / OPD_IQ | 8 | IQ 过流 | 电机 IQ 电流过大 | IQ 过流消除后执行清错指令即可 |
| OPD_U | 9 | U 相缺相 | 上电初始化时检测 | 检查电机 U 相线，恢复后执行清错指令 |
| OPD_V | 10 | V 相缺相 | 上电初始化时检测 | 检查电机 V 相线，恢复后执行清错指令 |
| OPD_W | 11 | W 相缺相 | 上电初始化时检测 | 检查电机 W 相线，恢复后执行清错指令 |
| POS_REF_OVER | 12 | 位置超上限 | 无 | 无 |
| POS_REF_UNDER | 13 | 位置超下限 | 无 | 无 |
| ABZ_ERR | 14 | 编码器异常 | 磁编一圈丢失计数 > 0.25% | 排查编码器，断电重启；持续报错需返厂 |
| ABZ_Z_ERR | 15 | 编码器 Z 相异常 | 初始化寻不到 Z 相 | 排查编码器，断电重启；持续报错需返厂 |
| ABZ_CALIB_ERR | 16 | 编码器校准错误 | 编码器线未接好或未校准 | 排查编码器或接线 |
| SPEED_REF_OVER | 17 | 给定速度超限 | 无 | 无 |
| CURRENT_REF_OVER | 18 | 给定电流超限 | 无 | 无 |
| JOINT_ENC_DISCONNECT | 19 | 电驱板上磁编断联 | 无 | 无 |
| MAGN2_NEGA_WARN | 27 | 磁编出现较小负值 | 磁编值小于 -180 大于 -360 | 无 |
| MAGN2_NEGA_ERR | 28 | 磁编出现较大负值 | 磁编值小于 -360 | 无 |
| NORM_POS_NEGA_WARN | 29 | 归一化行程较小负数 | 行程值 < 0 且 > -20 | 无 |
| NORM_POS_NEGA_ERR | 30 | 归一化行程较大负数 | 行程值 < -20 | 无 |
| DISCONNECT | 31 | 手指电机未连接 | 手指驱动与控制板通讯断开 | 连接后系统会自动清错并恢复 |

其余 RESERVED4 ~ RESERVED10 为保留 / 内部诊断位。



## 1.59 clearFaultCode

- **功能概述**

清除当前电机的故障码。

- **函数原型**

```
1  def clearFaultCode(self, slave_address: int) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

- True: 清除成功
- False: 清除失败

- **特别说明**

并非所有故障码都能被清除，详情参见各错误码的「解决办法」列。

## 1.60 getDeviceInfo

- **功能概述**

通过单次调用，统一获取从站设备的 SN 码、硬件版本、固件版本及 Modbus 配置等全部设备信息。

- **函数原型**

```
1  def getDeviceInfo(self, slave_address: int) -> tuple[bool, str, HardwareVersion, FirmwareVersion, ModbusConfig]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 tuple[bool, str, HardwareVersion, FirmwareVersion, ModbusConfig]:
- result[0]（bool）：是否获取成功
- result[1]（str）：设备 SN 码
- result[2]（HardwareVersion）：设备硬件版本信息
- result[3]（FirmwareVersion）：设备固件版本信息
- result[4]（ModbusConfig）：设备 Modbus 通信配置信息

## 1.61 getDeviceSN

- **功能概述**

  获取设备的 SN 序列号。

- **函数原型**

```
1  def getDeviceSN(self, slave_address: int) -> str
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

  返回设备 SN 码字符串（str）。

## 1.62 setTangentialForceCtlParam

- **功能概述**

  查询灵巧手的左右手类型。

- **函数原型**

```
1  def getHandType(self, slave_address: int) -> HandType
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

  返回 list[FingerMotorsError] 故障码列表, 每项字段:

| 枚举值 | 数值 | 说明 |
| --- | --- | --- |
| NONE_HAND | 0 | 无效灵巧手类型 |
| LEFT_HAND | 1 | 左手 |
| RIGHT_HAND | 2 | 右手 |



## 1.63 getFirmwareVersion

- **功能概述**

  获取设备的固件版本信息。

- **函数原型**

```
1  def getFirmwareVersion(self, slave_address: int) -> FirmwareVersion
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

  返回 FirmwareVersion 结构体, 当前仅字段 unified_version 可用。

## 1.64 getHardwareVersion

- **功能概述**

  获取设备的硬件版本信息。

- **函数原型**

```
1  def getHardwareVersion(self, slave_address: int) -> HardwareVersion
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

  返回 HardwareVersion 结构体, 当前仅字段 mainboard_hardware_version 可用。

## 1.65 getSDKVersion

- **功能概述**

  获取当前 SDK 的版本号。

- **函数原型**

```
1  def getSDKVersion(self) -> str
```

- **参数说明**

  无

- **返回值**

  返回 SDK 版本号字符串（str）。

## 1.66 upgradeFirmware

- **功能概述**

升级指定从站设备的固件。

- **函数原型**

```
1  def upgradeFirmware(self, slave_address: int, file_path: str) -> UpgradeErrorCode
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| file_path | str | 输入 | 固件文件路径 |

- **返回值**

返回 UpgradeErrorCode 枚举值, 常见取值:

| 枚举值 | 说明 |
| --- | --- |
| UPGRADE_SUCCESS | 升级成功 |
| UPGRADE_CHECK_OK | 升级检查通过 |
| UPGRADE_FAIL | 升级失败 |
| UPGRADE_REQUEST_ERROR | 升级请求错误 |
| UPGRADE_EXECUTE_ERROR | 升级执行错误 |
| UPGRADE_TYPE_ERROR | 升级类型错误 |
| FILE_COUNT_ERROR | 文件数量错误 |
| FILE_EXIT_ERROR | 文件不存在 |
| FILE_FORMAT_ERROR | 文件格式错误 |
| FILE_MATCH_ERROR | 文件匹配错误 |
| FILE_NAME_ERROR | 文件名错误 |
| FILE_SIZE_ERROR | 文件大小错误 |
| FILE_VERSION_ERROR | 文件版本错误 |
| FILE_HAND_TYPE_ERROR | 手型不匹配 |
| FILE_SEND_SUCCESS | 文件发送成功 |
| HAND_ENABLE_ERROR | 手部使能错误 |
| HAND_FAULT_ERROR | 手部故障 |
| PARAM_PART_ERROR | 部件参数错误 |
| SEND_EXTRA_ACTION_ERROR | 附加动作发送错误 |
| SEND_FILE_OPERATE_ERROR | 文件操作发送错误 |
| SEND_FRAME_1_ERROR | 第 1 帧发送错误 |
| SEND_FRAME_N_ERROR | 第 N 帧发送错误 |
| SEND_FRAME_LAST_ERROR | 末帧发送错误 |



## 1.67 upgradeSensor

- **功能概述**

升级传感器固件版本。开始升级前请重新给灵巧手上电。

- **函数原型**

```
1  def getFirmwareVersion(self, slave_address: int, info: list[UpgradeFileAndPart]) -> UpgradeErrorCode
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |
| info | list[UpgradeFileAndPart] | 输入 | 传感器固件与升级部位列表; 升级文件需要和程序放在同一目录下 |

- **UpgradeFileAndPart 结构说明**

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| file_path | str | 固件文件路径 |
| upgrade_part | list[int] | 需要升级的传感器部位编号集合 |

- **返回值**

返回 UpgradeErrorCode 枚举值, 常见取值 ( 同 1.66 节 upgradeFirmware ) :

| 枚举值 | 说明 |
| --- | --- |
| UPGRADE_SUCCESS | 升级成功 |
| UPGRADE_CHECK_OK | 升级检查通过 |
| UPGRADE_FAIL | 升级失败 |
| UPGRADE_REQUEST_ERROR | 升级请求错误 |
| UPGRADE_EXECUTE_ERROR | 升级执行错误 |
| UPGRADE_TYPE_ERROR | 升级类型错误 |

### 1.68 getUpgradeInfo

- **功能概述**

获取固件升级进度信息。如需使用，请在调用 upgradeFirmware 之前，另起一个独立线程循环调用本接口轮询进度。

- **函数原型**

```
1  def getFirmwareVersion(self, slave_address: int) -> list[UpgradeProcess]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| slave_address | int | 输入 | 从站地址 |

- **返回值**

返回 UpgradeProcess 列表，每个元素包含：

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| upgrade_type | FirmwareUpgradeType | 升级类型（枚举，见下表） |
| part | int | 升级的固件部位编号 |
| process_data | int | 升级进度值，范围 [0, 100] |
| status | str | 升级状态说明文字 |

- **FirmwareUpgradeType 枚举值**

| 枚举值 | 数值 | 说明 |
| --- | --- | --- |
| INVALID_UPGRADE | 0 | 无效的升级类型 |
| MAINBOARD_UPGRADE | 1 | 主控板升级 |
| DRIVER_UPGRADE | 2 | 电驱板升级 |
| SENSOR_UPGRADE | 3 | 传感器升级 |



## 2. 相机控制类 DexH15Camera

**负责灵巧手配套相机设备的连接、图像采集与参数配置。**
**构造函数：cam = DexH15Camera()，无参数。**

### 2.1 connectCameraDevice

- **功能概述**

打开并连接指定相机设备。

- **函数原型**

```
1  def connectCameraDevice(self, camera_port_num: str) -> None
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| camera_port_num | str | 输入 | 相机设备编号 |

- **返回值**

无返回值（None）。连接失败时抛出异常。

### 2.2 releaseCameraDevice

- **功能概述**

释放相机设备资源，断开连接。

- **函数原型**

```
1  def releaseCameraDevice(self) -> None
```

- **参数说明**

无

- **返回值**

无返回值（void）。

## 2.3 getFrame

- **功能概述**

从相机采集当前帧图像。

- **函数原型**

```
def getFrame(self) -> numpy.ndarray
```

- **参数说明**

无

- **返回值**

返回 numpy.ndarray 类型的图像帧( uint8 数组),默认分辨率 640×480,帧率 30 帧,若相机未打开,则返回空数组。

## 2.4 setCameraConfig

- **功能概述**

设置相机的分辨率与帧率参数。

- **函数原型**

```
def setCameraConfig(self, width: int, height: int, fps: float) -> bool
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| width | int | 输入 | 图像分辨率宽度 ( 像素 ) |
| height | int | 输入 | 图像分辨率高度 ( 像素 ) |
| fps | float | 输入 | 相机帧率 ( 帧 / 秒 ) |

- **返回值**

- True: 配置成功
- False: 配置失败

- **特别说明**

仅支持 640×480、1280×720、1920×1080 分辨率。帧率仅支持 640×480、1280×720 分辨率下的 30fps, 以及 1920×1080 分辨率下的 60fps。



## 2.5 showImage

- **功能概述**

在窗口中显示指定图像帧。

- **函数原型**

```
def showImage(self, imgName: str, img: numpy.ndarray) -> None
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| imgName | str | 输入 | 显示窗口的名称 |
| img | numpy.ndarray | 输入 | 要显示的图像 ( uint8 数组 ) |

- **返回值**

无返回值 ( None )。

# 3. 运动学解析类 DexH15Kinematic

**负责灵巧手关节角度与电机位置之间的正逆运动学转换，以及归一化角度与真实角度的互相换算。该类不依赖设备连接，可离线调用。**

**构造函数：kin = DexH15Kinematic()，无参数。类不依赖设备连接，可离线调用。**

## 3.1 forwardKinematicParse

- **功能概述**

  正运动学解析：输入 7 个电机位置值及 8 个磁编数据，计算并输出关节的真实角度值。当 return_joint_flag 为 False 时输出 11 个可调关节角度；为 True 时额外输出 4 个末端指骨（DIP）关节，共输出 15 个关节角度。

- **函数原型**

```python
def forwardKinematicParse(
    self,
    hand_type: HandType,
    motor_pos: list[int],
    magnetic_encoder: list[int],
    hardware_version: str = ""
    return_joint_flag: bool = False
) -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
|---|---|---|---|
| hand_type | HandType | 输入 | 左手或右手类型 |
| motor_pos | list[int] | 输入 | 7 个电机位置值集合，取值范围 [0, 10000] |
| magnetic_encoder | list[int] | 输入 | 8 个磁编码器数据集合 |
| hardware_version | str | 输入 | 硬件版本（可选，默认为空字符串） |
| return_joint_flag | bool | 输入 | 是否返回全部 15 个关节，默认 False（返回 11 个可调关节） |

- **返回值**

  返回 tuple[int, list[float]]:
  - result[0] int: 1 成功 / 0 失败
  - result[1] list[float]return_joint_flag=False 时长度为 11,True 时长度为 15(关节顺序角度范围同 C++ 版 3.3 节)



## 3.2 inverseKinematicParse

- **功能概述**

  **逆运动学解析：输入 11 个可调关节的真实角度值，计算并输出对应的 7 个电机位置值。**

- **函数原型**

```python
def inverseKinematicParse(
    self,
    hand_type: HandType,
    real_angle: list[float],
    hardware_version: str = ""
) -> tuple[int, list[int]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
|---|---|---|---|
| hand_type | HandType | 输入 | 左手或右手类型 |
| real_angle | Sequence[float] | 输入 | 11 个可调关节的真实角度值集合（单位：°） |
| hardware_version | str | 输入 | 硬件版本（可选，默认为空字符串） |

- **返回值**

  返回 tuple[int, list[int]]:
  - result[0]（int）：状态码，1 解析成功 / 0 解析失败
  - result[1]（list[int]）：解析得到的 7 个电机位置值

### 3.3 calculateRealAngle

- **功能概述**

  **根据 11 个可调关节的归一化角度集合，计算对应的真实物理角度（单位：度）。**

- **函数原型**

```
1  def calculateRealAngle(self, normalize_angle: list[float], hardware_version: str = "") -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| normalize_angle | Sequence[float] | 输入 | 11 个关节的归一化角度集合 |
| hardware_version | str | 输入 | 硬件版本（可选，默认为空字符串） |

- **返回值**

  返回 tuple[int, list[float]]:
  - result[0]（int）：状态码，1 计算成功 / 0 计算失败
  - result[1]（list[float]）：计算得到的 11 个真实角度集合（单位：°）

- **特别说明**

  与控制类 calculateRealAngle 的区别：运动学类可独立串口控制使用，并支持指定 hardware_version 的灵巧手使用。

### 3.4 calculateNormalizeAngle

- **功能概述**

  **根据 11 个可调关节的真实物理角度，计算对应的归一化角度集合。**

- **函数原型**

```
1  def calculateNormalizeAngle(self, real_angle: list[float], hardware_version: str = "") -> tuple[int, list[float]]
```

- **参数说明**

| 参数名 | 类型 | 输入 / 输出 | 说明 |
| --- | --- | --- | --- |
| real_angle | list[float] | 输入 | 11 个关节的真实角度集合（单位：°） |
| hardware_version | str | 输入 | 硬件版本（可选，默认为空字符串） |

- **返回值**

  返回 result: tuple[int, list[float]]:
  - result[0]（int）：状态码，1 计算成功 / 0 计算失败
  - result[1]（list[float]）：计算得到的 11 个归一化角度集合



### 3.5 getSDKVersion

- **功能概述**

  **获取当前 SDK 的版本号。**

- **函数原型**

```
1  def getSDKVersion(self) -> str
```

- **参数说明**

  **无**

- **返回值**

  返回 SDK 版本号字符串（str）。

## 4. 关节顺序定义

**数组均为 11 个可调关节，顺序固定为：食指（2）→ 中指（2）→ 无名指（2）→ 小拇指（2）→ 大拇指（3）。**

| 索引 | 关节标识 | 对应手指 | 归一化范围 | 真实角度范围（°） | 说明 |
| --- | --- | --- | --- | --- | --- |
| 0 | index_joint1 | 食指 | [0, 1000] | [0, 80] | 近端关节，弯曲伸展 |
| 1 | index_joint2 | 食指 | [0, 1000] | [0, 80] | 末端关节，弯曲伸展 |
| 2 | middle_joint1 | 中指 | [0, 1000] | [0, 80] | 近端关节，弯曲伸展 |
| 3 | middle_joint2 | 中指 | [0, 1000] | [0, 80] | 末端关节，弯曲伸展 |
| 4 | ring_joint1 | 无名指 | [0, 1000] | [0, 80] | 近端关节，弯曲伸展 |
| 5 | ring_joint2 | 无名指 | [0, 1000] | [0, 80] | 末端关节，弯曲伸展 |
| 6 | little_joint1 | 小拇指 | [0, 1000] | [0, 80] | 近端关节，弯曲伸展 |
| 7 | little_joint2 | 小拇指 | [0, 1000] | [0, 80] | 末端关节，弯曲伸展 |
| 8 | thumb_joint1 | 大拇指 | [-500, 500] | [0, 109.6] | 掌指双轴关节，侧位摆动 |
| 9 | thumb_joint2 | 大拇指 | [0, 1000] | [0, 87] | 近端关节，弯曲伸展 |
| 10 | thumb_joint3 | 大拇指 | [0, 1000] | [0, 120.1] | 末端关节，弯曲伸展 |

# 5. 数据结构

## 5.1 Dex15MotorPosition

- 7 路电机位置值。构造：Dex15MotorPosition()。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| motor1_pos ~ motor7_pos | int | 电机 1~7 位置 |

## 5.2 Dex15MotorCurrent

- 7 路电机电流值。构造：Dex15MotorCurrent()（统一初始值）。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| motor1_current ~ motor7_current | int | 电机 1~7 电流 |

## 5.3 Dex15MotorSpeed

- 7 路电机速度值。构造：Dex15MotorSpeed()（统一初始值）。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| motor1_speed ~ motor7_speed | int | 电机 1~7 速度 |

## 5.4 ForcePoint

- 触觉力点。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| x / y / z | int | 坐标位置 |

## 5.5 PressForcePoint

- 按压位置力点。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| x / y / z | int | 坐标位置 |
| force_value | int | 力值大小 |
| model_name | str | 模型名称 |

## 5.6 FingerTemp

- 手指温度信息。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| finger_name | str | 手指名称 |
| sensor_ID | int | 传感器 ID |
| temperature | float | 温度值（℃） |

## 5.7 FingerMotorsError

- 手指电机故障信息。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| error_code | ErrorCode | 错误码 |
| finger_name | str | 手指名称 |
| motor_ID | int | 电机 ID |

## 5.8 ForceCtlParam

- 普通力控参数。

| 字段 | 类型 | 范围（单个关节） | 说明 |
| --- | --- | --- | --- |
| enable_flag | list[bool] | True/False | 各手指力控使能标志 |
| impedance_factor | list[float] | [1, 100] | 阻抗系数 |
| reference_force_factor | list[float] | [1, 10] | 参考力系数 |

## 5.9 TanForceCtlParam

- 切向力力控参数（在普通力控参数基础上增加切向增益）。

| 字段 | 类型 | 范围（单个关节） | 说明 |
| --- | --- | --- | --- |
| enable_flag | list[bool] | True/False | 各手指力控使能标志 |
| impedance_factor | list[float] | [1, 100] | 阻抗系数 |
| reference_force_factor | list[float] | [1, 10] | 参考力系数 |
| tangential_force_gain | float | [1, 100] | 切向力增益 |

## 5.10 ModbusConfig

- Modbus 通信配置。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| baud_rate | int | 波特率 |
| slave_id | int | 从机 ID |

## 5.11 HardwareVersion

- 硬件版本信息。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| driver_hardware_version | list[MotorVersion] | 各电机驱动板硬件版本 |
| mainboard_hardware_version | str | 主控板硬件版本 |

### 5.12 FirmwareVersion

- 固件版本信息。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| unified_version | str | 统一固件版本 |
| driver_version | list[MotorVersion] | 该字段不可用 |
| mainboard_version | str | 该字段不可用 |
| sensor_version | list[SensorVersion] | 该字段不可用 |

### 5.13 MotorVersion

- 单个电机版本信息。

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| part | int | 部件编号 |
| part_name | str | 部件名称 |
| version | str | 版本号 |

### 5.14 固件升级相关结构

| 结构 | 字段 | 说明 |
| --- | --- | --- |
| FirewareFields | reserve1~3、upgrade_type、upgrade_version | 固件文件字段信息 |
| UpgradeFileAndPart | file_path、upgrade_part: set[int] | 升级文件与部件对应关系 |
| UpgradeInfo | file_fields、file_path、upgrade_part | 升级信息（含文件字段解析结果） |
| UpgradeProcess | part、process_data、status、upgrade_type | 升级进度信息 |
| SensorSequence | part、part_name、sequence | 传感器序列信息 |
| SensorVersion | part、part_name、version | 传感器版本信息 |



## 6. 枚举类型

### 6.1 ControlMode（电机控制模式）

| 成员 | 说明 |
| --- | --- |
| NONE | 无模式（获取失败时返回） |
| CURRENT_CONTROL_MODE | 电流控制 |
| VELOCITY_CONTROL_MODE | 速度控制 |
| POSITION_CONTROL_MODE | 位置控制（默认） |

### 6.2 HandType（手型）

| 成员 | 说明 |
| --- | --- |
| NONE_HAND | 无效手型 |
| LEFT_HAND | 左手 |
| RIGHT_HAND | 右手 |

### 6.3 FingerType（手指类型）

| 成员 | 说明 |
| --- | --- |
| INVALID_FINGER | 无效 |
| INDEX_FINGER | 食指 |
| MIDDLE_FINGER | 中指 |
| RING_FINGER | 无名指 |
| PINKY_FINGER | 小拇指 |
| THUMB_FINGER | 大拇指 |

### 6.4 FingerMotor（电机编号）

| 成员 | 数值 | 说明 |
| --- | --- | --- |
| INVALID_MOTOR | 0 | 无效 |
| INDEX_MOTOR | 1 | 食指电机 |
| MIDDLE_MOTOR | 2 | 中指电机 |
| RING_MOTOR | 3 | 无名指电机 |
| PINKY_MOTOR | 4 | 小拇指电机 |
| THUMB_MOTOR_1 | 5 | 大拇指侧摆电机 |
| THUMB_MOTOR_2 | 6 | 大拇指电机 2 |
| THUMB_MOTOR_3 | 7 | 大拇指电机 3 |

PaXini



# 帕西尼感知科技


电话：**帕西尼·深圳**：0755-23574593

**帕西尼·上海**：021-54561536

**帕西尼·天津**：022-82411882

邮箱：**mkt@paxini.com**

官网：**www.paxini.com**

地址：**帕西尼感知科技（深圳）有限公司**：广东省深圳市宝安区海城路 5 号前城中心 34F-35F

**帕西尼感知科技研究分部（深圳）**：广东省深圳市南山区万科云城六期一栋云中城

**帕西尼感知科技智能实验 & 制造中心（深圳）**：广东省深圳市宝安区永丰社区

**帕西尼感知科技（上海）有限公司**：上海市徐汇区桂平路 391 号新漕河泾国际商务中心

**帕西尼（天津）Super EID Factory**：天津市河西区天津空天信息产业园

**帕西尼感知科技制造研发中心（苏州）**：江苏省苏州市张家港市经开区华夏科技园


**京东 POP 店**

**天猫旗舰店**

**京东自营店**

**关注我们 了解更多**